

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Sistemi Operativi

Appunti a cura di
Lorenzo Ferraro

2 ottobre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Sistemi Operativi*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@gmail.com**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Introduzione	1
1.1	Architettura dell'Hardware	1
1.1.1	Von Neumann Architecture	1
1.1.2	Ciclo delle istruzioni	3
1.1.3	CPU	4
1.1.4	CPU Registers	4
1.1.5	Instruction Pipelining	5
1.1.6	CPU Performance	7
1.1.7	I/O	11
1.2	Introduzione ai sistemi operativi	13
1.2.1	Cos'è il sistema operativo?	13
1.2.2	Astrazioni	15
1.2.3	Dual-Mode	17
1.2.4	Chiamate di sistema	18

Capitolo 1

Introduzione

1.1 Architettura dell'Hardware

1.1.1 Von Neumann Architecture

Prima di entrare nel vivo del corso, è bene fare un'introduzione riguardo l'architettura dell'hardware. Questo perché i sistemi operativi si pongono come **ponte** tra hardware e software.

Definizione 1.1 – Von Neumann Architecture

L'**Architettura Von Nuemann** è il fondamento teorico di tutte le architetture hardware dei computer moderni. Si basa su diversi principi:

- Le istruzioni e i dati **si trovano nello stesso spazio in memoria**
- Le istruzioni sono **sequenziali**: vengono lette una ad una dalla memoria per poi essere eseguite dalla CPU

Le componenti principali che la compongono sono:

- **Processing Unit**, dove si trovano la **ALU** (un circuito digitale dove vengono svolti i calcoli algebrici più semplici) e i **registri** (piccolissime e velocissime unità di memoria volatile)
- **Control Unit**
- **Memoria RAM** dove si trovano i dati e le istruzioni
- **Mass Storage / External I/O**
- **System Bus**, il quale collega tutti gli altri componenti e li fa lavorare insieme

La Control Unit e la Processing Unit si trovano nella CPU, e comunicano con la RAM e con i dispositivi I/O tramite il System Bus.

Osservazione 1.1 – Von Neumann Bottleneck

Un classico problema dell'Architettura Von Neumann si verifica quando la RAM e la CPU comunicano: essendo la CPU molto più veloce, quando queste lavorano la RAM non riesce a tenere il passo, e dato che la CPU ha bisogno dei dati che si trovano in RAM, si ritrova spesso ad **aspettare inutilmente**, rendendo questa architettura non sempre efficiente.

Per risolvere questo problema è importante cercare di non accedere alla memoria se non strettamente necessario, sfruttando la cache ad esempio.

Definizione 1.2 – Harvard Architecture

Esiste un'altra scuola di pensiero riguardo l'architettura hardware: l'**Architettura Harvard**. La differenza più importante rispetto all'Architettura Von Neumann è che qui i dati e le istruzioni **non si trovano nello stesso spazio in memoria**.

Definizione 1.3 – System Bus

Il **System bus** (o Internal Bus) è l'insieme di cavi che permette alla CPU, alla memoria e all'I/O di comunicare fra loro. Questo si divide in 3 componenti principali:

- **Address Bus**, che contiene l'indirizzo di memoria del dato richiesto dalla CPU
- **Data Bus**, che trasporta effettivamente i dati
- **Control Bus**, che trasporta il comando da eseguire sul dato

L'**Address Bus** è **unidirezionale**, dato che viaggia sempre dalla CPU verso la memoria e mai il contrario. La **Bus width** è il numero linee fisiche che compongono il bus. Con N linee si riesce ad ottenere un numero massimo di 2^N locazioni uniche. Fino a qualche anno fa erano comuni i sistemi a 32 bit, ma essendo diventati troppo limitanti si è passati ai modelli a 64 bit. Questi, in realtà, sono troppo anche per gli hardware moderni, e dunque per risparmiare si usano spesso dei modelli fisici a 48 bit, molto più che sufficienti.

Il **Data Bus** è **bidirezionale**, dato che il dato può sia arrivare alla CPU (se lo si vuole leggere), sia partire da essa (se lo si vuole scrivere). La **Bus width** indica quanti bit si possono trasferire in un singolo clock cycle. La **Word Size** invece è la quantità di bit con cui la CPU elabora i dati (da cui si dividono i processori a 32 bit o a 64 bit).

Il **Control Bus** può trasportare diversi messaggi, tra cui i più comuni sono:

- **MEMR/MEMW** (Memory Read/Memory Write), che permettono di leggere/scrivere dati in memoria
- **IOR/IOW** ((I/O Read)/(I/O Write)), che permettono di comunicare con i dispositivi I/O
- **Bus Reques/Bus Grant**, che permettono al DMA controller di usare il Bus

- **Interrupt Signals**, messaggi mandati dai componenti hardware di notificare la CPU di un evento.

1.1.2 Ciclo delle istruzioni

Definizione 1.4 – Ciclo delle istruzioni

Il **ciclo delle istruzioni** è alla base di ogni operazione di un computer, che lo esegue fin dall'avvio. Si divide in tre passaggi:

- 1° **Fetch**
- 2° **Decode**
- 3° **Execute**

Fetch

Durante la fase di Fetch, la CPU copia il valore del **PC** (Program Counter), che contiene l'indirizzo di memoria dove si trova la prossima istruzione da eseguire, nel **MAR** (Memory Address Register) e lo inserisce anche nell'address bus. A questo punto la Control Unit manda un comando di Memory Read al control bus. Si legge dunque l'indirizzo di memoria dell'istruzione e i dati interessati da questa istruzione vengono inseriti nel data bus. La CPU li legge e li inserisce nel **IR** (Instruction Register). Infine, il PC viene incrementato così da contenere al suo interno la prossima istruzione.

Decode

Durante la fase di decodifica, la Control Unit legge i bit che si trovano nell'Instruction Register. Identifica l'**Opcode** (Operation Code), ossia l'operazione da eseguire che può essere ad esempio ADD o LOAD, e gli **Operands**, ossia i registri, gli indirizzi di memoria o i valori semplici su cui eseguire tale operazione. A questo punto sempre la Control Unit prepara la ALU e i registri per eseguire tale operazione.

Execute

Durante la fase di esecuzione, in base al tipo di istruzione vi sono tre possibili scenari:

- 1) Se l'istruzione è aritmetica, come ad esempio l'istruzione `add %eax, %ebx` (vedremo ogni tanto del codice assembly, non è essenziale ma può aiutare a capire di cosa si sta parlando. In questo caso, stiamo facendo una somma tra i valori contenuti all'interno di due registri, `eax` e `ebx`), parliamo di **ALU operations**, dato che è la ALU che se ne occupa, facendo l'operazione e scrivendo il risultato finale.

- 2) Se l'istruzione è una load/store, come ad esempio l'istruzione `mov (%rax), %rcx` (stiamo muovendo il valore del registro `%rax` nel registro `%rcx` performando dunque un'operazione di load), parliamo di **Memory operations**, dove la CPU legge/scrive dati nella RAM usando MAR o MDR.
- 3) Se l'istruzione è un jump/branch, come ad esempio l'istruzione `jmp 0x4010a0` (stiamo saltando all'indirizzo `0x4010a0`), parliamo di **Control flow**, dove il Program Counter viene aggiornato direttamente all'indirizzo indicato, modificando il flow di esecuzione.

1.1.3 CPU

Definizione 1.5 – CPU

La **CPU** è il cervello del computer, che esegue le operazioni in memoria. Questo è diviso in tre blocchi principali:

- **Datapath**, che contiene i registri e la ALU
- **Control Unit**, che prende le istruzioni e le decodifica, e in base a queste gestisce il datapath
- **Internal Bus**, che collega i registri, la ALU e la CU (Control Unit)

La **Datapath** performa le operazioni aritmetiche (somma, sottrazione, moltiplicazione e divisione) e **bitwise** (ADD, OR, XOR, NOT e gli shift), grazie alla ALU, e ha al suo interno i **GPRs** (General-Purpose Registers), che sono delle celle di memoria interne alla CPU per mantenere dati così da non dover sempre accedere alla memoria, risparmiando tempo.

La **Control Unit** gestisce il datapath e gli dice **come** e **quando** agire. Prende in input l'Instruction Register, dei segnali di clock o dei flags, e in base a questi manda in output segnali diversi alla ALU, ai registri, o ai sistemi di I/O. È il coordinatore della CPU.

1.1.4 CPU Registers

Abbiamo visto nel datapath i registri, che sono piccolissime e velocissime unità di memoria nella CPU. Si dividono in due macrogruppi:

- **User-visible registers** (ossia i **General-Purpose**), che sono richiamabili direttamente tramite assembly (come i registri `%rax` e `%rbx` visti prima)
- **Control & Status registers** (ossia i **Special-Purpose**), che sono usati dalla Control Unit per modificare il flusso di esecuzione

General-Purpose Registers

In base al tipo di CPU questi registri sono salvati con nomi differenti:

- Per le CPU con architettura x86-64 vi sono 16 registri chiamati %rax, %rbx, %rcx, %rdx, %rsi, %rdi, %rdp, %rsp e poi da %r8 fino a %r15
- Per le CPU con architettura ARM64 vi sono 31 registri che vanno da x0 a x30

Special-Purpose Registers

Vi sono molti Special-Purpose Registers, come ad esempio il Program Counter, l'Instruction Register, lo Stack Pointer, il MAR e l'MDR e il PSR. Vediamoli più nello specifico:

- Il **Program Counter**, come già detto, contiene l'indirizzo di memoria per la prossima istruzione da eseguire.
- L'**Instruction Register** contiene l'istruzione da eseguire.
- Lo **Stack Pointer** mantiene l'indirizzo di memoria della cima della call stack
- Il **MAR** (Memory Address Register) contiene l'indirizzo che sta venendo letto o scritto sulla RAM. È strettamente collegato all'address bus.
- L'**MDR** (Memory Data Register, o anche detto MBR, ossia Memory Buffer Register) contiene i dati letti dalla RAM o che devono esser scritti sulla RAM. È strettamente collegato al data bus.
- Il **PSR** (Program Status Register) contiene i bit flag che indicano lo stato della CPU e l'ultima operazione effettuata dall'ALU. Tra le flag più note ci sono:
 - **Zero Flag (Z)**, che diventa 1 se l'ultima operazione dell'ALU ha restituito 0 come valore
 - **Sign/Negative Flag (S/N)**, che diventa 1 se l'ultimo risultato era negativo

Queste flag sono fondamentali per eseguire if e cicli: infatti, per controllare ad esempio se due valori x e y siano uguali, il computer effettua una sottrazione e verifica se il risultato sia zero grazie proprio alla Zero Flag.

1.1.5 Instruction Pipelining

Definizione 1.6 – Pipelining

Nei processori più semplici, ogni istruzione deve essere presa, decodificata e eseguita prima di poter passare alla prossima. Questo però non è molto efficiente, e per questo oggi si utilizza una tecnica nota come **pipelining** dove più istruzioni vengono eseguite contemporaneamente.

Proprietà 1.1 – RISC 5-Stage Pipeline

Una pipeline standard si divide in **5 stage**:

- 1° **IF** (Instruction Fetch): si prende l'istruzione
- 2° **ID** (Instruction Decode): si decodifica l'istruzione
- 3° **EX** (Execute): si esegue l'istruzione
- 4° **MEM** (Memory Access): si legge o si scrivono i dati dell'operazione (eseguita solo se necessaria)
- 5° **WB** (Writeback): scrive il risultato nel registro di destinazione

Proprietà 1.2 – Pipeline Hazards

Tramite il Pipelining l'output ideale sarebbe di eseguire un'istruzione per clock cycle, ma questo è spesso quasi impossibile perché vi sono diversi **hazards** che rallentano la pipeline. Si dividono in:

- **Structural Hazards**, quando due istruzioni diverse hanno bisogno dello stesso hardware fisico e dunque non possono essere eseguite contemporaneamente.
- **Data Hazards**, quando un'istruzione dipende dal risultato di un'altra e dunque deve aspettare che questa finisca.
- **Control Hazards**, quando il PC viene modificato da un jump/branch, modificando il flusso di esecuzione e rendendo le istruzioni prese inutili. Per risolvere tali problemi molte CPU moderne utilizzano i **branch predictors**, che provano a indovinare il risultato di un branch. Possono essere molto utili, ma quando questi dovessero sbagliare si perderebbe ancora più tempo del normale.

Definizione 1.7 – Instruction Set Architectures

La **ISA** (Instruction Set Architectures) definisce l'interfaccia tra hardware e software. Vi sono due possibili scelte molto diverse:

- **CISC** (Complex Instruction Set Computer), che ha una lunga lista di istruzioni già pronte, come ad esempio gli Intel x86/x86-64
- **RISC** (Reduced Instruction Set Computer), che ha invece un set di istruzioni più semplici, come ad esempio gli ARM (Apple M-series)

1.1.6 CPU Performance

Definizione 1.8 – Processori multicore

Per superare i limiti di potenza, si è pensato di **usare più processori (core)** su un singolo dispositivo.

Ogni core è una CPU indipendente che può far partire il proprio ciclo di istruzioni indipendentemente dalle altre.

Definizione 1.9 – Clock

Ogni CPU lavora in base al proprio **System Clock**. Esiste il **Clock period**, che è la durata di un singolo clock cycle (come ad esempio 0.25 ns), e la **Clock Frequency**, ossia il numero di cicli di Clock al secondo, misurato in Hz e che si calcola banalmente prendendo il reciproco del periodo.

Le istruzioni più semplici impiegano anche un solo ciclo, ma quelle più complesse possono impiegare anche molti. Esiste quindi il **CPI**, ossia il numero di cicli medio per istruzione, e anche il **IPC**, ossia il numero di istruzioni per ciclo (che non è altro che l'inverso del CPI). Avendo più core, i processori moderni riescono nelle condizioni ideali anche a superare il limite di un istruzione per ciclo, ottenendo quindi un valore di $IPC > 1$.

Osservazione 1.2 – Memory wall

Negli anni, i processori sono migliorati in modo esponenziale, al contrario della DRAM, che ha avuto un miglioramento molto più lineare. Questo gap ha portato alla creazione di un **memory wall**: la CPU si ritrova spesso ad aspettare la RAM che è troppo lenta. Abbiamo già visto in breve questo problema quando abbiamo parlato del Von Neumann Bottleneck.

Per risolvere questo problema, si è creata una **struttura gerarchica delle memorie**:

- 1° **Registri** (tempo: $< 1ns$, capacità: $< 1KiB$)
- 2° **L1 Cache** (tempo: $1ns$, capacità: $32/64KiB$)
- 3° **L2 Cache** (tempo: $3ns$, capacità: $256/512KiB$)
- 4° **L3 Cache** (tempo: $10ns$, capacità: $8/64MiB$)
- 5° **RAM** o Memoria Principale (tempo: $60ns$, capacità: $8/64GiB$)
- 6° **SSD** (tempo: $10\mu s$, capacità: $256GiB/2TiB$)
- 7° **HDD** (tempo: $10ms$, capacità: $1/10TiB$)

È evidente che i registri sono molto più veloci di un SSD o della memoria RAM, ma sono anche meno capienti, e più costosi. Infatti, il costo di tutte queste memorie diminuisce scendendo giù nella gerarchia, rendendo l'HDD il meno costoso e i registri i più costosi.

Principio di località

Proprietà 1.3 – Principio di località

Quando si accede a un indirizzo in memoria, è molto probabile che bisognerà accedervi anche nel breve futuro. Per questo, si salva queste informazioni in memoria cache per potervi accedere più velocemente senza dover tornare nella RAM. Questa viene detta **località temporale**.

Allo stesso modo, se si accede a un indirizzo in memoria, è molto probabile che bisognerà accedere anche agli indirizzi a lui vicino. Per questo, si salvano anche questi nella cache. Questa viene detta **località spaziale**.

Prendiamo questo codice in C come esempio:

```
1 int sum = 0;
2 for (int i = 0; i < 1000; i++){
3     sum += array[i];
4 }
```

Per la località temporale, le variabili `sum` e `i` sono salvate nei registri o nella L1 cache, così da non dover riaccedere alla DRAM. Allo stesso modo, per la località spaziale, è molto probabile che anche i valori `array[1]`, `array[2]`... saranno richiesti in un breve futuro, dunque la CPU salva quanti più possibili di questi valori riempiendo una linea di L1 cache, di 64 bytes. Dato che ogni `int` in C occupa 4 byte, si salveranno in totale 16 valori dell'array, da `array[0]` a `array[15]`. In questo modo accediamo una sola volta alla DRAM al posto di 16.

RAM

Definizione 1.10 – RAM

La memoria **RAM** si divide in due sottocategorie:

- 1) **SRAM** (Static RAM), veloce ma costosa. Usata per le cache delle CPU
- 2) **DRAM** (Dynamic RAM), lenta ma più economica. Usata per la memoria principale (quando si parla di RAM si intende quasi sempre la DRAM)

Fisicamente, la RAM è un **vettore lineare di locazioni**. Nella maggior parte delle architetture l'indirizzamento è al byte: a ogni indirizzo univoco corrisponde esattamente un singolo byte (8 bit). La CPU può comunque richiedere accessi a parole di 4 o 8 byte, accedendo a una sequenza di indirizzi contigui.

Cache

Definizione 1.11 – Linee di cache

Le cache non operano sui singoli byte, ma su strutture chiamate **linee di cache**. Questi sono blocchi (di solito di 64 byte), che contengono:

- Un **valid bit**, che indica se i dati salvati sono validi o meno
- I **tag bit**, utile per identificare la singola linea
- I **data block**, ossia gli effettivi 64 byte copiati dalla RAM

Quando la CPU richiede un accesso in memoria, ci sono due possibilità:

- **Cache Hit**: la CPU trova l'indirizzo nel giro di 1/2 cicli.
- **Cache Miss**: la CPU non trova l'indirizzo nella cache, dunque è costretta a cercarla nella DRAM, arrivando a impiegare fino a 100 cicli di clock.

Cerchiamo di calcolare la AMAT (Average Memory Access Time), ossia il tempo medio per accedere a un indirizzo di memoria.

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Più il Miss Rate aumenta più questo valore aumenta, rendendo l'intero processo più lento.

Osservazione 1.3 – Cache Coherence Problem

Nei sistemi multicore, ogni core ha un suo L1 cache. Questo però crea un **problema di coerenza**: se il Core 0 e il Core 1 hanno lo stesso valore X, ma il core 1 lo aggiorna modificandone il valore, il core 0 non avrà il valore aggiornato.

Per risolvere questo problema vi sono i **coherence protocols**, che invalidano o aggiornano le copie nelle altre cache ogni volta che si presenta questo problema.

Proprietà 1.4 – Cache Write Policies

Quando la CPU scrive sulla cache, vi sono due modi per aggiornare anche la RAM con il nuovo valore:

- **Write-Through Policy:** la CPU scrive in contemporanea sia sulla Cache che sulla RAM. Dunque la RAM è sempre aggiornata, ma ogni operazione di scrittura diventa più lenta dovendo ogni volta accedere alla RAM.
- **Write-Back Policy:** la CPU aggiorna solo la Cache line, che viene marcata come *Dirty*, e la RAM viene aggiornata solo quando la cache va eliminata. Dunque le operazioni di memoria sono veloci, ma la RAM contiene valori vecchi.

Cache Line Mapping Policies

Vi sono diversi modi per mappare un blocco RAM a una specifica sezione della cache:

- 1) **Direct Mapped**
- 2) **Fully Associative**
- 3) **Set Associative**

Il **Direct Mapped** è l'approccio più semplice: l'indice della cache viene mappato usando l'aritmetica modulare:

$$\text{Cache Slot} = \text{Block Address} \% \text{Numero di slot della cache}$$

Se il numero di slot per la cache è 4, avremo quindi che i blocchi 0, 4, 8, ... andranno allo slot 0, quelli 1, 5, 9, ... allo slot 1 e così via. In questo caso vi è un problema chiamato **cache thrashing**: se vi sono due blocchi che devono andare allo stesso slot, è possibile che questi ogni volta si rubino il posto a vicenda. Se ad esempio inizialmente serve il blocco 0, questo andrà nello slot cache 0. Quando serve il blocco 4, anche questo deve andare nello slot 0, dunque viene tolto il blocco 0 per far spazio al blocco 4. Se poi servirà nuovamente il blocco 0, bisognerà togliere il blocco 4 per fargli spazio e così via.

Il **Fully Associative** è invece l'approccio più libero: ogni blocco può andare dove vuole. Non vi è alcun cache thrashing, ma richiede ricerche in parallelo per controllare ogni singola linea di cache finché non si trova quella che serve. Questo è molto costoso e lento, rendendo questa tecnica utile solo per strutture molto piccole.

Il **Set Associative** è un compromesso tra il Direct Mapped e il Fully Associative. La cache viene organizzata in N insiemi, che contengono un valore di blocchi prestabilito (ad esempio, in un 4-way Set Associative ogni insieme ha 4 blocchi). Un blocco viene poi associato a uno di questi insiemi tramite l'aritmetica modulare:

$$\text{Set Index} = \text{Block Address} \% \text{Numero insiemi}$$

Questo riduce notevolmente il cache thrashing (dato che vi è spazio per più di un blocco allo stesso indice), e allo stesso tempo mantiene velocità accettabili.

Memoria Secondaria

Vi sono due tipi di memoria secondaria:

- **HDD** (Hard Disk Drivers), ossia il disco meccanico. Questo è molto lento perché è necessario che un braccio meccanico fisico si muova sopra la traccia, ruoti, e legga/scriva sulla superficie meccanica. Il sistema operativo in questo caso deve fare il massimo per limitare i movimenti meccanici del disco.
- **SSD** (Solid State Drivers), che invece non ha parti meccaniche. I dati qui sono strutturati in **pagine** che vengono riunite in **blocchi** di, ad esempio, 128 pagine. Si può leggere tranquillamente ogni singola pagina, ma per riscriverla bisogna cancellare l'intero blocco a cui appartiene, il che rende tutto molto più lento dato che bisognerà poi ricopiare l'intero blocco. Ogni blocco, poi, può supportare solo un numero limitato di cancellazioni prima che si degradi. Il sistema operativo deve in questo caso mandare dei comandi TRIM per aiutare il controller dell'SSD a gestire i blocchi vuoti.

1.1.7 I/O

Un computer ha senso solo se comunica con strumenti esterni. Vi sono ad esempio i **dispositivi di storage**, come ad esempio gli SSD o gli HDD, o le **interfacce utente**, come la tastiera o il mouse. Indipendentemente dal tipo, ogni dispositivo di I/O è gestito da due layer:

- **Device Controller** (Hardware), un microchip che espone dei nuovi registri alla CPU da poter controllare
- **Device Driver** (Software), un modulo nel sistema operativo che dice alla CPU come comunicare con tale dispositivo.

La CPU, per comunicare con il Device Controller, ha due possibilità:

- **PMIO** (Port-Mapped I/O), delle istruzioni della CPU dedicate all'I/O. Ad esempio, nelle CPU con architettura x86, vi sono i comandi in e out per riferirsi in modo chiaro ai dispositivi I/O.
- **MMIO** (Memory-Mapped I/O), dove i registri del device di I/O vengono mappati direttamente nella memoria fisica, potendo così utilizzare dei normali puntatori per accedervi.

Ad oggi, l'MMIO è lo standard.

Proprietà 1.5 – Registri del Device Controller

Nel Device Controller vi sono 3 tipi di registri:

- **Status Register:** read-only dato che riporta lo status del dispositivo
- **Command Register:** write-only dato che contiene le istruzioni per il dispositivo
- **Data Register:** read/write dato che contiene i dati mandati o ricevuti

I/O Transfer Protocols

La CPU ha tre diversi protocolli per comunicare con un dispositivo I/O:

- **Programmed I/O (Polling)**
- **Interrupt-Driven I/O**
- **DMA (Direct Memory Access)**

Il **Programmed I/O**, più comunemente chiamato **Polling**, è il metodo più semplice: semplicemente, se la CPU vuole ad esempio leggere qualcosa su un dispositivo I/O, legge lo Status Register del suo Device Controller, e se lo stato è "Busy" continua a chiederlo, finché non lo è più e può finalmente leggere i dati dal Data Register. Dunque semplicemente la CPU continua a chiedere al dispositivo i dati finché questo non è libero e può darglieli. Questo processo prende il nome di *busy-waiting*. Il problema più grande di questo protocollo è che la CPU è impegnata al 100%.

L'**Interrupt-Driven I/O** risolve il problema del busy-waiting: semplicemente, sono direttamente i dispositivi di I/O ad avvertire la CPU di esser pronti. Dunque la CPU manda la richiesta al device, e mentre aspetta esegue altre istruzioni. Quando il dispositivo manda l'ok, inviando un segnale su una **Interrupt Line**, la CPU mette in pausa l'esecuzione, e passa al **IVT** (Interrupt Vector Table), un array di puntatori che gli indica dove trovare la corretta **ISR** (Interrupt Service Routine), che è una speciale funzione che salva tutti i flags e i registers e li ripristina alla fine dell'esecuzione dell'istruzione. Vi sono due problemi con questo protocollo:

- salvare/ripristinare i registri e i flags richiede tempo
- se si ha un dispositivo molto veloce, è possibile che la CPU sia sempre interrotta perché le istruzioni iniziate finiscono subito, finendo così per utilizzarne il 100% solo per spostarsi da un'istruzione all'altra.

Il **DMA** (Direct Memory Access) consiste nel delegare il trasferimento ad un controller hardware a parte, chiamato appunto **DMA controller**. Se per esempio bisogna spostare 4 KiB dal Disco alla RAM, la CPU può delegare tale operazione al DMA controller che lo farà senza dover coinvolgere la CPU. Alla fine, il DMA controller crea un singolo segnale che invia alla CPU

per avvisare della fine del trasferimento. Il più grande problema del DMA è che condivide il bus di sistema con la CPU, dunque se la CPU deve accedere alla RAM spesso finisce per dover aspettare il DMA controller.

La soluzione dunque è quella di sfruttare le potenzialità di tutti e 3 i protocolli, usando uno o l'altro in base alla situazione. Ad esempio, per operazioni veloci e semplici il polling è la migliore opzione, mentre per dispositivi lenti, come la tastiera, l'Interrupt-Driven I/O ha più senso.

1.2 Introduzione ai sistemi operativi

1.2.1 Cos'è il sistema operativo?

Definizione 1.12 – Sistema Operativo

Un **sistema operativo** (OS) è una collezione di sistemi software con l'obiettivo di gestire le risorse hardware e offrire dei servizi alle applicazioni.

Si può considerare il sistema operativo come il primo software che il computer avvia e che viene eseguito in una modalità privilegiata che prende il nome di **modalità kernel**, che rivedremo più avanti.

I vantaggi del sistema operativo sono molteplici:

- Senza di esso, dovremmo scrivere applicazioni che si occupano ad esempio anche di scrivere e leggere i dati direttamente sul disco o di gestire direttamente gli indirizzi della RAM. Con esso invece viene creata un'astrazione più semplice e pulita che tutte le app possono usare:
 - non ci sono più settori numerati sulla memoria secondaria ma **file** e **cartelle**
 - non c'è più memoria RAM da gestire direttamente ma ci sono **spazi di indirizzamento virtuali**
 - non ci sono più le interfacce network ma ci sono i **socket**
- La gestione delle risorse del computer è lasciata a lui. Decide quali programmi possono usare la CPU e per quanto tempo, non lascia nessuna applicazione senza risorse per troppo tempo e ottimizza questo utilizzo per minimizzare gli sprechi.
- Gestisce, inoltre, tutti i programmi in esecuzione. Isola le diverse applicazioni, in modo tale che due applicazioni si ostacolino a vicenda, previene che queste modificano codice o impostazioni di sistema e, se un'applicazione crasha, evita di far crashare tutto il sistema.

Definizione 1.13 – Kernel

Il **kernel** (nucleo) è la componente centrale del sistema operativo. È il primo software caricato dal computer ed è sempre attivo.

Proprietà 1.6 – Obiettivi sistema operativo

Gli obiettivi del sistema operativo sono molti:

- **Virtualizzazione**, ossia la pratica di far apparire le risorse fisiche come delle risorse logiche
- **Concorrenza**, ossia il saper gestire contemporaneamente più processi senza problemi mantenendo i dati consistenti
- **Persistenza**, ossia l'abilità di salvare i dati su dei sistemi progettati per il lungo termine (dato che la RAM è volatile, bisogna salvare spesso i dati e in modo veloce sulla memoria secondaria)
- **Sicurezza e isolamento**, ossia l'abilità di gestire la sicurezza dei file e gli spazi di memoria dei diversi processi
- **Efficienza**, ossia l'abilità di dare priorità ad aspetti diversi in base alla circostanza

Esistono due tipi di virtualizzazione:

- **CPU Virtualization**, dove l'OS trasforma una singola CPU fisica in molte CPU virtuali (detti processi o *thread*, che vedremo meglio tra poco)
- **Memory Virtualization**, dove l'OS dà a ogni processo il suo personale spazio di memoria virtuale da poter usare

Per quanto riguarda invece la sicurezza, come già accenato prima è necessario che ogni programma abbia il suo spazio di memoria virtuale a cui gli altri non possono accedere, e se un singolo programma crasha deve crashare solo lui e non tutto il sistema. Inoltre, in un sistema multi-user, per i file protetti è necessario controllare che l'utente che vuole accedervi abbia l'autorizzazione necessaria per poterlo fare.

Infine, per quanto riguarda l'efficienza, è importante capire che il SO fa sempre dei "*trade-off*", ossia deve scegliere se dare priorità a una cosa piuttosto che a un'altra. La domanda più classica è: "Velocità o sicurezza?" Purtroppo è necessario scegliere: se infatti si vuole la sicurezza, allora bisognerà aumentare i controlli e dunque la velocità diminuirà, altrimenti se si vuole aumentare quest'ultima il nostro sistema diventerà più vulnerabile.

Storia del sistema operativo

I primi computer negli anni 40 e 50 non avevano neppure un sistema operativo. Si usavano delle schede perforate dove veniva scritto il codice binario, si runnava il codice e lo si debuggava sul momento. La potenza della CPU era praticamente inespressa: doveva infatti rimanere ad aspettare un sacco di tempo l'input umano.

Alla fine degli anni 50 si vede un **primo prototipo di sistema operativo**: infatti i programmatori potevano caricare più programmi da eseguire insieme, e il sistema operativo si sarebbe occupato di gestirne il flusso: caricava il primo programma, lo eseguiva, e poi caricava il secondo e così via. La CPU, però, rimaneva assai limitata dal sempre troppo lento input umano.

Per eliminare questo spreco di tempo per la CPU, si è pensato negli anni 60 di far lavorare la CPU anche mentre aspetta un input umano. Ad esempio, se il programma 1 aveva bisogno di un input, la CPU non avrebbe più aspettato inutilmente ma avrebbe iniziato a eseguire il programma 2. Questo aumentava molto l'efficienza, ma anche il carico sul sistema operativo, che ora doveva isolare la memoria di più programmi e doveva gestire lo switch tra un programma e l'altro della CPU.

Negli anni 70 si iniziano a vedere i primi sistemi di **timesharing** (come UNIX): la CPU, tramite un clock interno, cicla tra i diversi utenti attivi che possono interagire con la macchina tramite un proprio terminale. Dato che lo switch tra le macchine avviene in millisecondi, all'utente sembra di avere la macchina tutta per sé, potendo quindi usare una sola CPU in contemporanea tra più persone senza alcun delay notevole. Questo crea ancora più pressioni sul sistema operativo che deve ora molti nuovi problemi.

Dagli anni 80 ad oggi abbiamo visto sempre più sistemi: negli anni 80 arrivano i PC (Personal Computers), con però diversi problemi di sicurezza, come MS-DOS, mentre dagli anni 90 ad oggi sono arrivati i sistemi operativi che tutti conosciamo, come Windows o macOS. Per i telefoni vi sono invece iOS e Android, che prioritizzano l'efficienza elettrica e la sicurezza al resto.

1.2.2 Astrazioni

Definizione 1.14 – Astrazione

Un'**astrazione** è una vista semplificata di un sistema complesso che espone solo alcuni dettagli mentre nasconde le meccaniche più complesse.

Definizione 1.15 – Processo

L'astrazione più importante del sistema operativo è il **processo**, che è un'**istanza di un programma**.

Per capire bene cos'è il processo, basti pensare che il programma è il codice statico che si trova sulla memoria secondaria che noi scarichiamo, mentre un processo rappresenta il programma in esecuzione in un determinato momento.

Definizione 1.16 – Thread

Dentro un programma possono esserci dei **thread**, che è la più piccola unità di esecuzione gestibile in modo autonomo dal SO.

Tutti i thread di uno stesso processo hanno lo stesso spazio di memoria e gli stesso file, ma ogni thread ha un proprio *PC* (dato che ogni thread può eseguire azioni indipendentemente dagli altri, ha bisogno di un proprio Program Counter che gli dice quali istruzioni eseguire), stack e un registro di stato.

Astrazione della memoria Principale

La nostra RAM fisica è una griglia di celle: per gestire meglio questo spazio, il sistema operativo la astrae in **spazio di indirizzamento**. Ogni processo ha un range di indirizzi che va da 0 a 2^N-1 in base al sistema (nei sistemi a 64 bit si arriva fino a $2^{64}-1$ indirizzi diversi). Ogni indirizzo virtuale viene poi mappato ad un indirizzo fisico unico. Due processi A e B possono avere quindi lo stesso indirizzo di memoria virtuale 0x1000 (nel proprio spazio di indirizzamento) mappato a due indirizzi fisici differenti (rispettivamente 0x7fff000 e 0x8aaa000, ad esempio).

Questa gestione della memoria è tutta opera del sistema operativo, e viene fatta per tre motivi:

- **Facilità d'uso:** è molto più facile assumere che il primo indirizzo in memoria sia 0 e non un indirizzo fisico reale
- **Isolazione:** in questo modo se un programma sbaglia qualche reference e crasha, crasha solo lui e non tutto il sistema
- **Efficienza:** il sistema operativo carica solo le parti attualmente utili del processo, così da risparmiare risorse

Astrazione della memoria Secondaria

Le memorie secondarie operano su blocchi fisici e settori. Per rendere questi blocchi di dati più comodi il sistema operativo li astrae in **file**, ossia degli array di bytes a cui viene dato un nome e che possono essere letti o scritti. Per organizzare questi file vi sono poi le **cartelle**, ossia dei file speciali che contengono file o altre cartelle. Si creano dunque un *albero gerarchico delle cartelle*, dove se una cartella contiene un'altra significa che si trova nella gerarchia a un livello più in alto. Sarà poi il sistema operativo a trasformare i percorsi come /home/Utente/file.txt in degli indirizzi fisici di memoria.

1.2.3 Dual-Mode

Il sistema operativo, essendo solo software, ha bisogno di un supporto hardware per evitare che l'utente possa toccare sezioni del computer che non lo riguardano. Per questo motivo, le CPU moderne hanno due diverse modalità di operazioni:

- **modalità utente** (non privilegiata), usata per eseguire le applicazioni dell'utente e che non ha accesso a tutto
- **modalità kernel** (privilegiata), usata per eseguire codice del sistema operativo e che ha accesso completo a tutte le istruzioni della CPU

Nelle architetture x86 in realtà oltre alla modalità utente e la modalità kernel vi sono due modalità intermedie per i driver dei dispositivi. Questi quattro livelli sono chiamati **anelli**, ma nella pratica i SO moderni utilizzano solo l'anello 0 (modalità kernel) e l'anello 3 (modalità utente), spostando i driver o in uno o nell'altro in base alla situazione. Questo rende i sistemi operativi più **semplici e chiari**, e soprattutto li rende più **accessibili** a diverse CPU, che ad oggi supportano solo l'utilizzo di due livelli.

Per sapere in che modalità ci troviamo, la CPU usa un **bit modalità** che si trova in un registro di stato, e che vale **0** per la **modalità kernel** e **1** per la **modalità utente**. Quando si vuole eseguire un'istruzione, la CPU controlla prima se nella modalità attuale questa istruzione è eseguibile o meno.

Definizione 1.17 – Istruzioni

Le **istruzioni privilegiate** sono le istruzioni che la CPU può eseguire solo se si trova nella modalità kernel. Alcune istruzioni privilegiate sono ad esempio lo spegnimento della CPU, oppure modificare il puntatore del MMU o mandare segnali a dei registri hardware. Le **istruzioni non privilegiate** sono invece le istruzioni che la CPU può eseguire sempre, anche in modalità utente. Alcune istruzioni non privilegiate sono ad esempio le istruzioni aritmetiche (add, sub, mult, div) o le istruzioni logiche (and, or, xor).

Se ad esempio la CPU in modalità utente decodifica un'istruzione non privilegiata, la CPU non la esegue e l'hardware triggerà un **GPF** (General Protection Fault) o un'Eccezione. La CPU switcha automaticamente alla modalità kernel e passa al gestore delle eccezioni del kernel, che termina il processo.

Per cambiare dalla modalità utente a quella kernel ci sono solo 3 modi:

- **Chiamate di sistema / traps**: l'utente richiede al sistema operativo di eseguirgli per lui un'istruzione privilegiata
- **Eccezioni**: il programma dell'utente fa qualcosa di illegale
- **Interruzioni hardware**: un dispositivo esterno manda un segnale di interruzione

Per cambiare da quella kernel a quella utente invece il kernel deve prima salvare tutte le informazioni per poter poi eseguire una serie di istruzioni in modo *atomico* (ossia deve eseguire un set di istruzioni tutte insieme, se una fallisce nessuna deve essere eseguita):

- 1° Ripristina il PC
- 2° Imposta il bit di modalità a 1
- 3° Riprende le istruzioni del programma

1.2.4 Chiamate di sistema

Un programma per funzionare ha però bisogno di eseguire delle istruzioni privilegiate (altrimenti non potrebbe salvare nulla in memoria, ad esempio). Per questo nascono le **chiamate di sistema**.

Definizione 1.18 – Chiamate di sistema

Le **chiamate di sistema** (**system calls**, syscalls o traps) sono delle richieste che le applicazioni fanno al kernel per eseguire istruzioni privilegiate.

A differenza delle chiamate di libreria, che si possono eseguire interamente in modalità utente e sono molto veloci, le chiamate di sistema sono significativamente più lente perché coinvolgono anche il kernel. Esistono, però, delle chiamate di libreria dette **wrappers**, le quali al loro interno eseguono una chiamata di sistema. Spesso nel codice non si utilizzano le system calls in modo diretto ma dei wrappers, così da non dover scrivere in assembly (linguaggio delle chiamate di sistema).

Quando ad esempio un'applicazione chiama una funzione `read()`, succedono diverse cose:

- 1° Il wrapper copia gli argomenti negli specifici registri della CPU
- 2° Il numero identificativo della chiamata di sistema viene inserito in `%rax`
- 3° Il wrapper esegue la syscall
- 4° La CPU prende dal vettore delle syscall la chiamata che gli serve, cambia la modalità in modalità kernel, e va all'indirizzo del gestore centrale delle syscall, ossia il punto di ingresso unico per tutte le syscall
- 5° Il kernel valida gli argomenti, esegue la syscall, e mette il valore di ritorno in `%rax`, ed esegue `sysret`, che permette di ritornare rapidamente alla modalità utente

Il wrapper ha tre metodi per poter passare i parametri al kernel:

- **Registri**
- **Puntatore stack**: se i registri sono pieni si passa a questi solo un puntatore che punta ad un blocco di memoria

- **Stack utente:** gli argomenti sono passati allo stack utente

La regola numero uno riguardo le chiamate di sistema è quella di **non fidarsi mai dell'input dell'utente**. Infatti, prima di far partire una syscall la CPU deve sempre validare i parametri della chiamata. Deve controllare se i puntatori puntano a spazi di memoria validi o se il processo ha i permessi necessari per fare quella determinata azione ad esempio.

Perché le chiamate di sistema sono così costose per il computer?

Abbiamo visto come le chiamate di sistema siano decisamente più lente delle chiamate di libreria. Questo succede per diversi motivi:

- Bisogna passare dalla modalità utente alla modalità kernel e nuovamente alla modalità utente
- Bisogna salvare tutti i parametri della chiamata nei registri o nei vari stack, e se vi sono puntatori che puntano a blocchi di memoria secondaria questi possono rallentare di molto l'operazione
- Bisogna pulire la cache e i registri ogni volta così da averli pronti per le prossime chiamate

Per questo si cerca di limitare la frequenza delle system calls tramite un processo che prende il nome di **buffering**.